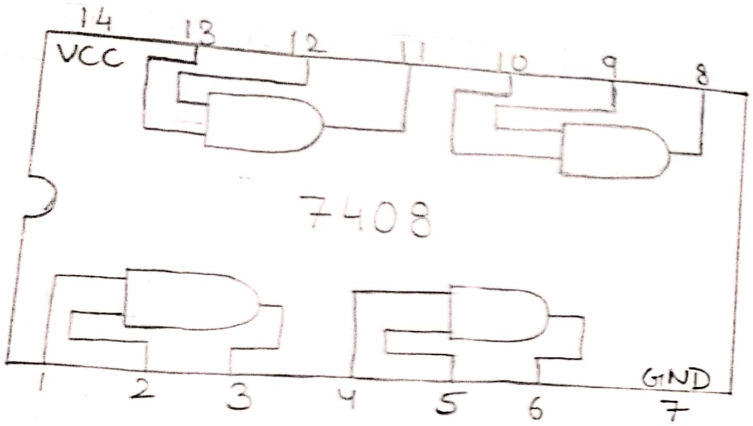



AND GATE

EXP-1

DATE 05/03/25
PAGE 1

## Experiment No. 1

OBJECTIVE:

To verify the truth table of AND, OR, NOT, NAND, NOR, EXOR using two input gates.

EQUIPMENT REQUIRED:

| S.NO | NAME OF COMPONENTS | SPECIFICATIONS | QUANTITY |
|---|---|---|---|
| 1 | Digital Trainer Kit | DL-ADTK | 1 |
| 2 | IC 7400, 7402 | NAND, NOR | 1 |
| 3 | IC 7404, 7408 | NOT, AND | 1 |
| 4 | IC 7432, 7486 | OR, EX-OR | 1 |
| 5 | Patch chords / connecting wires | - | As per requirement |

THEORY:

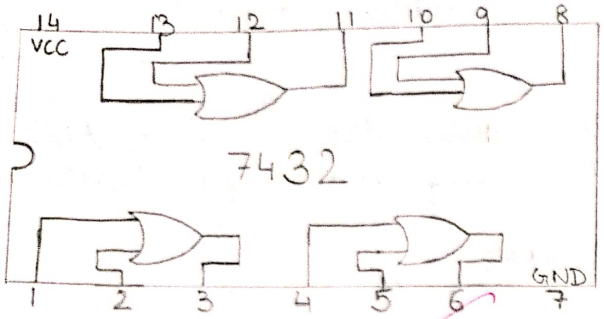
The the truth table of basic gates are shown below

1) AND GATE

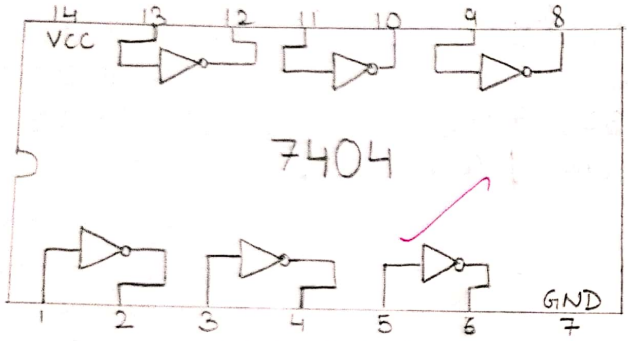
| A | B | A • B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

2) OR GATE

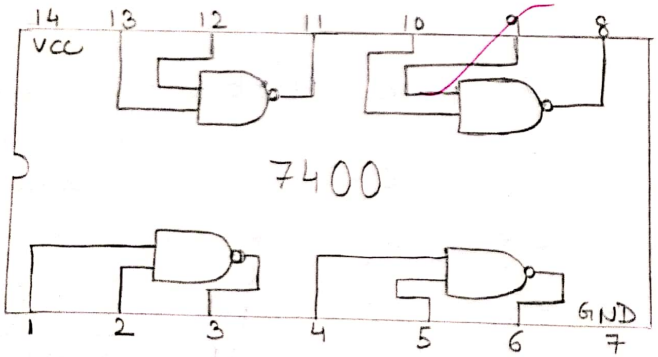
Scanned with OKEN Scanner




OR GATE




NOT GATE




NAND GATE

DATE
PAGE 2.

| A | B | A+B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

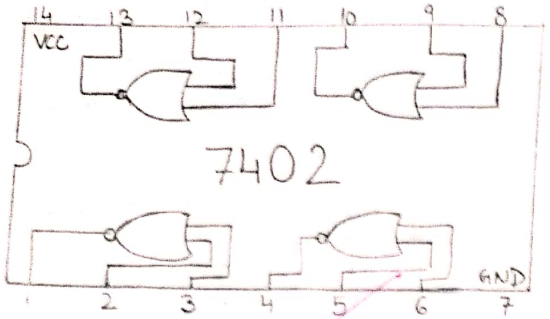
3) NOT GATE

| A | $\overline{A}$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

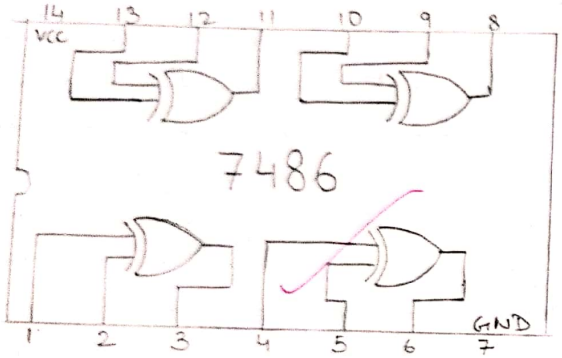
4) NAND GATE

| A | B | $\overline{A \cdot B}$ |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 0 |

Scanned with OKEN Scanner




NOR GATE




EX-OR GATE



5) NOR GATE

| A | B | $\overline{A+B}$ |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

6) EX-OR GATE

| A | B | $(A \oplus B)$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

RESULT:

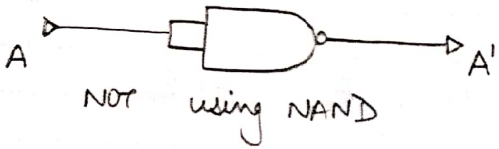
The truth table of corresponding gates is verified on digital trainer kit using ICs.

PRECAUTIONS:

1. All connections should be according to circuit diagram
2. All connections should be right & tight.
3. Power supply should be switched off after completing the experiment

Scanned with OKEN Scanner

1.






# Experiment No. 2

OBJECTIVE:

To design basic gates using universal gates.

EQUIPMENT REQUIRED:

| S.No | NAME OF COMPONENTS | SPECIFICATIONS | QUANTITY |
|---|---|---|---|
| 1. | Digital trainer kit | DL-ADTK | 1 |
| 2. | IC 7400, 7402 | NAND, NOR | 5 each |
| 3. | Patch chords/ connecting wires | - | As per requirement |

THEORY:

NOT, AND, OR gates are called basic gates because use of their different combinations, we can make different gates like NAND and NOR gates but not the other way, the NAND & NOR gates are universal gates as by using one of them, we are able to make any logic circuits.

(a) NOT using NAND

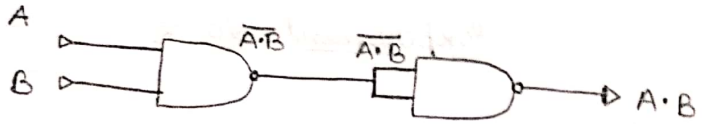
By short circuiting the input of NAND, we get the NOT gate, it is so because from the input then AB is the output.

- Boolean expression :-

$$Y = A = A \cdot A = \overline{A \cdot A} = \overline{A}$$

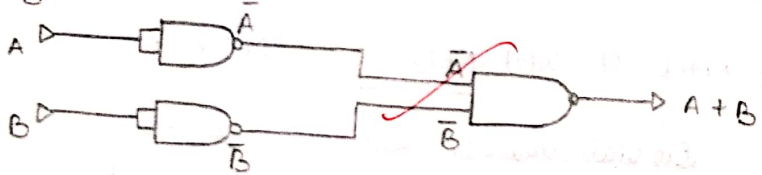
Scanned with OKEN Scanner

2.




AND using NAND

3.



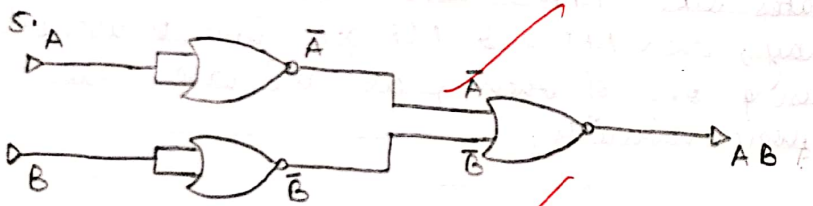

OR using NAND

4.



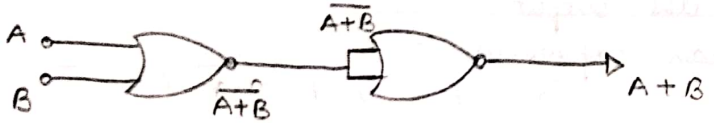

NOT using NOR

5.




AND using NOR

6.




OR using NOR

DATE
PAGE 5

(b) AND using NAND

The logic circuit for this combination is given, as we need two NAND gates to form an AND gate.

- Boolean expression :-

$$Y = \overline{\overline{A \cdot B}} = AB$$

(c) OR using NAND

The logic is, we need 2 NAND gates in parallel and a NAND gate is in series with them.

- Boolean expression :-

$$Y = \overline{\bar{A} \cdot \bar{B}} = \bar{\bar{A}} + \bar{\bar{B}} = A + B$$

(d) NOT using NOR.

By short circuiting the input of NOR gate, we get NOT gate as shown from the Boolean expression, if A and B are inputs, then $\overline{A+B}$ is the output. If A is shorted, the $\bar{A}$ is the output.

(e) AND using NOR

The circuit made of 2 short circuited NOR gates in parallel and again connected with NOR gates.

- Boolean expression :-

$$Y = \overline{\bar{A} + \bar{B}} = \bar{\bar{A}} \cdot \bar{\bar{B}} = A \cdot B$$

(f) OR using NOR.

This circuit is drawn using NOR gate in series with a short circuited NOR gate.

- Boolean expression

$$Y = \overline{\overline{A+B}} = A + B$$

Scanned with OKEN Scanner

PROCEDURE:

1. Connections are made as per circuit diagram.
2. Logical inputs are given as per circuit diagram.

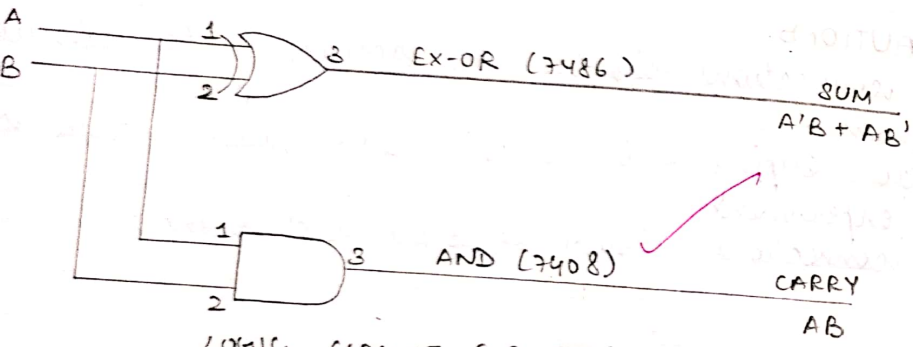
RESULT:

Verified truth tables for the gates using universal gates.

PRECAUTIONS:

1. All connections should be according to circuit diagram.
2. Power supply switched off after and before completing the experiment.
3. All connections should be tight and right.

Scanned with OKEN Scanner




LOGIC CIRCUIT FOR HALF ADDER



## Experiment No. 03

**OBJECTIVE:**

To design and implement Half Adder & Full Adder using logic gate.

**EQUIPMENT REQUIRED:**

| S.No. | Name of components | Specifications | Quantity |
|---|---|---|---|
| 1. | Digital Trainer kit | DL-ADTK | 1 |
| 2. | IC 7404, 7408 | NOT, AND | 3 |
| 3. | IC 7432, 7486 | OR, EX-OR | 2 |
| 4. | Patch chords/Connecting wires | | As per reqd. |

**THEORY:**

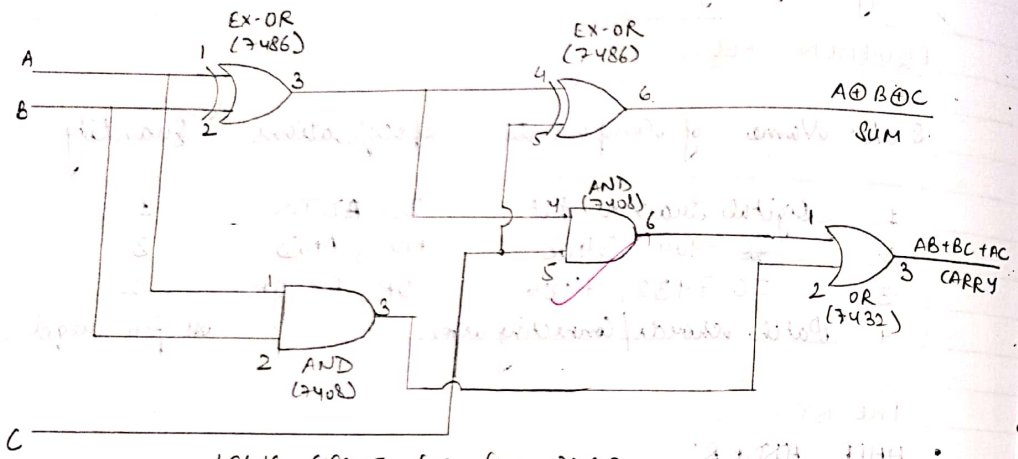
- **HALF ADDER:**

This means we have to add two bits with the possibility of a carry. The circuit used for this is called half adder. the O/P of the exclusive EX-OR gate is called the sum and output of AND gate is the carry. The AND gate produce a high output only when both are high. The exclusive EX-OR gate produce high O/P if either input, but not both is high.

→ Truth table

P.T.O →

Scanned with OKEN Scanner




LOGIC CIRCUIT FOR FULL ADDER



| A | B | CARRY | SUM |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

- FULL ADDER:

For the high order column, we have to use full adder circuit that can added 3 bits at a time. The third bit is the carry from a lower column. When two or more of input A, B and C is high, it will generate carry. Also, when an odd number of high A, B, and C arrives the EX-OR gate, it produces high o/p. The o/p of EX-OR gate is called SUM and the output OR gate is called gate.

→ Truth table

| A | B | C | CARRY | SUM |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

Scanned with OKEN Scanner

PROCEDURE:

1. Connections are given as per circuit diagram.
2. Logical inputs are given as per circuit diagram.
3. Observe the output and verify the truth table.

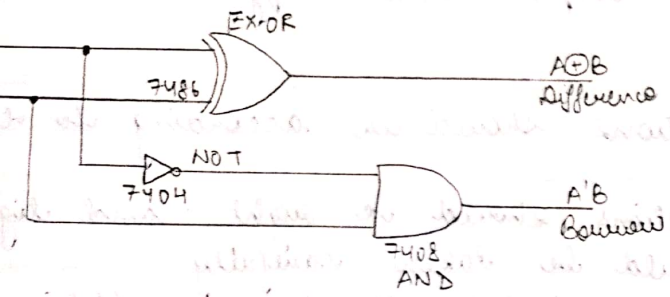
PRECAUTIONS:

1. All connections should be according to circuit diagram.
2. All connections should be right and tight.
3. Reading should be taken carefully.
4. Power supply switched off after completing the exp.

RESULT:

Operation of adder/ logic circuit is verified.

Scanned with OKEN Scanner




HALF SUBTRACTOR



# Experiment No. 4

OBJECTIVE:
To design and construct half subtractor and full subtractor using logic gates

EQUIPMENT REQUIRED:
Bread board, connecting wires, power supply (+5V), LED display board, IC- 7408, 7486, 7404, 7432 etc.

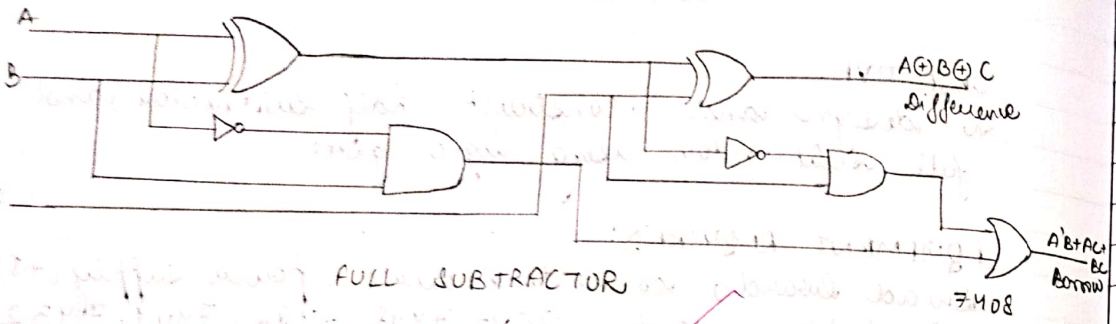
THEORY:

* HALF SUBTRACTOR:
The half subtractor is connected using EX-OR and AND Gate. The half subtractor has 2 input and 2 outputs. The outputs are difference and borrow. The difference can be applied using X-OR gate, borrow o/p can be implented using an AND gate and an inverter.

Truth table:

| A | B | BORROW | DIFFERENCE |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 0 |

Scanned with OKEN Scanner




FULL SUBTRACTOR

DATE
PAGE 11

* FULL SUBTRACTOR :

The full subtractor is a combination of X-OR, AND, OR, NOT, gates. In a full subtractor the logic circuit should have three inputs and two output. The two ray subtractor will be C and AB. The output will be difference O/P of full subtractor. The expression AB resembles the borrow output of the Ray subtractor and the second term is the inverted difference O/P of first X-OR.

Truth Table :

| A | B | C | BORROW | DIFFERENCE |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

PROCEDURE :

1. Connections are given per circuit diagram.
2. Logical inputs are given as per circuit diagram.
3. Observe the O/P and verify the truth table.

Scanned with OKEN Scanner

PRECAUTIONS:

1. All the connections should be according to circuit diagram.
2. All the connections should be ~~a~~ right and tight
3. ~~R~~ Reading should be taken carefully
4. Power supply switched off after completing the exp.

RESULT:

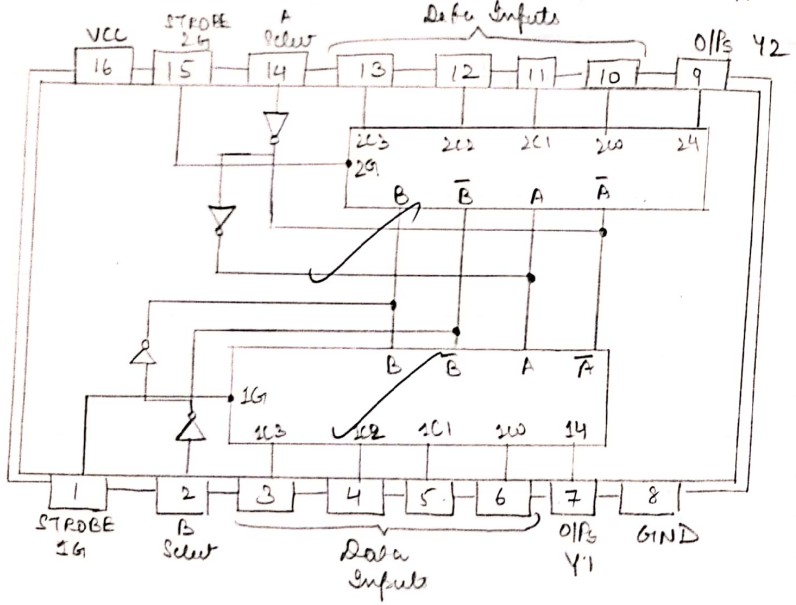
Operation of subtractor circuit is verified

| Select lines | Input | | O/P |
|---|---|---|---|
| S | $D_0$ | $D_1$ | Y |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Truth Table for 2:1 MUX

| Select Inputs | | Enable Input | Inputs | | | | O/P |
|---|---|---|---|---|---|---|---|
| $S_1$ | $S_0$ | E | $I_0$ | $I_1$ | $I_2$ | $I_3$ | Y |
| X | X | 1 | X | X | X | X | 0 |
| 0 | 0 | 0 | 0 | X | X | X | 0 |
| 0 | 0 | 0 | 1 | X | X | X | 1 |
| 0 | 1 | 0 | X | 0 | X | X | 0 |
| 0 | 1 | 0 | X | 1 | X | X | 1 |
| 1 | 0 | 0 | X | X | 0 | X | 0 |
| 1 | 0 | 0 | X | X | 1 | X | 1 |
| 1 | 1 | 0 | X | X | X | 0 | 0 |
| 1 | 1 | 0 | X | X | X | 1 | 1 |

Truth Table for 4:1 MUX




IC 74153 MUX

EXP-5

DATE 09/09/2025
PAGE 13

# Experiment No. 5

OBJECTIVE:

Design and ~~simulate~~ implementation of Multiplexer and Demultiplexer

EQUIPMENT REQUIRED:

| S.No. | Name of Component | Specifications | Quantity |
|---|---|---|---|
| 1. | Digital trainer kit | DL- ADTK | 1 |
| 2. | IC 7400, 7408 (for implementation using NAND gates) | NAND | |
| 3. | IC 74153, IC 74139 | MUX & DEMUX | 1+1 as per req'd. |
| 4. | Patch chords / connecting wire. | | |

Bread board, connecting wires, power supply (+5 volt), LED display, IC-7486

THEORY:

- Multiplexers:

They are very useful components in digital systems. They transfer a larger number of information units over a smaller number of channels, (usually one channel) under the control of selection signals. Multiplexer means ~~a~~ many to one. A multiplexer is a circuit with many inputs but only one O/P. By using control signals (select lines) we can select any input to the O/P. Multiplexer is also

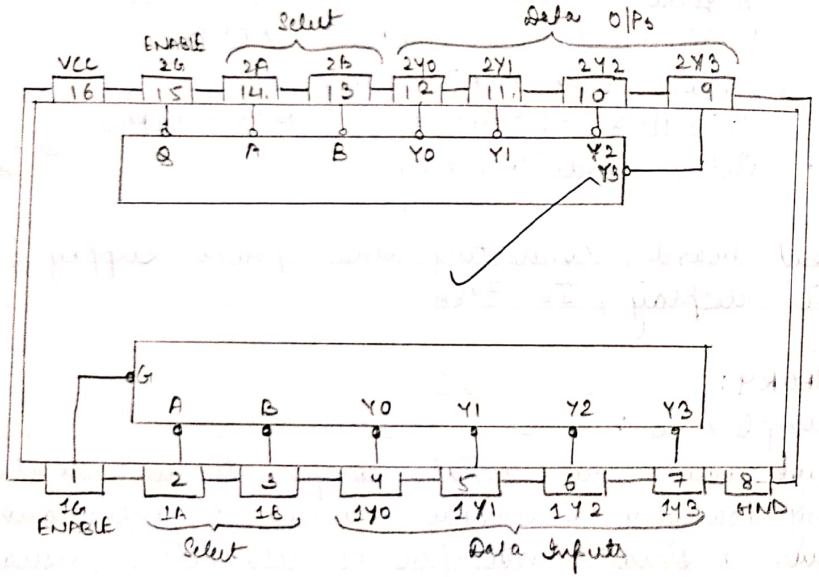
Scanned with OKEN Scanner

| INPUT | OUTPUT | |
|---|---|---|
| $S_0$ | $Y_1$ | $Y_0$ |
| 0 | 0 | A |
| 1 | A | 0 |

Truth table for 1:2 DEMUX

| Enable Inputs | Data Input | Select Inputs | | Outputs | | | |
|---|---|---|---|---|---|---|---|
| E | D | $S_1$ | $S_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
| 1 | 0 | X | X | X | X | X | X |
| 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |

Truth Table for 1:4 DEMUX




IC 74139

DE-MUX



called as data selector because the O/P line depend on the input data line that is selected. The general multiplexer circuit has $2^n$ inputs; n control select signals and 1 O/P signal.

- De-multiplexers:

Perform the opposite function of multiplexers. They transfer a small number of information units (usually one unit) over a larger number of channel under the control of selection signals. The general de-multiplexer circuit has 1 I/P signal, n control / select signal and $2^n$ O/P signals. De-multiplexer circuit can also be realised using a decoder circuit with enable.

PROCEDURE:

1. Connections are given as per circuit diagram
2. Logical inputs are given as per circuit diagram.
3. Observe the O/P and verify the truth table.

RESULT:

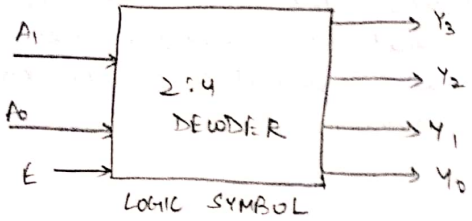
The MUX and DEMUX designed are verified.

PRECAUTIONS:

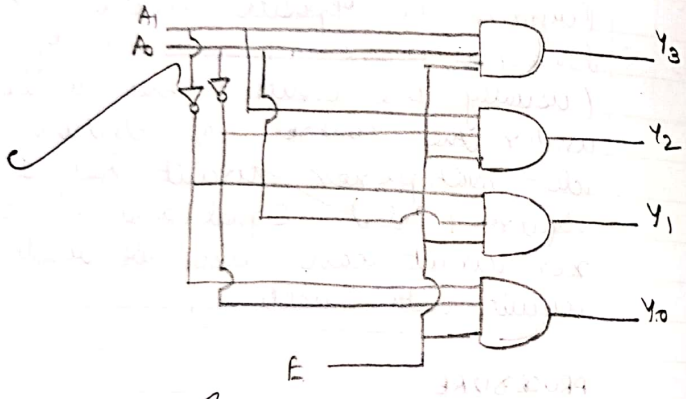
1. Insert IC carefully in the base without damaging the pins.
2. Take care while supplying the voltages to the IC.
3. Keep in mind input and output pins of the IC while performed the experiment.

## 2x4 DECODER

enable 1




LOGIC SYMBOL




$Y_3 = E \cdot A_1 \cdot A_0$

$Y_2 = E \cdot A_1 \cdot \overline{A_0}$

$Y_1 = E \cdot \overline{A_1} \cdot A_0$

$Y_0 = E \cdot \overline{A_1} \cdot \overline{A_0}$

Truth Table:

| ENABLE | INPUTS | | OUTPUTS | | | |
|---|---|---|---|---|---|---|
| E | $A_1$ | $A_0$ | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
| 0 | x | x | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |

EXP-06

DATE 16/09/2022
PAGE 15

# Experiment No. 06

**OBJECTIVE:**
Design and implementation of encoder and decoder

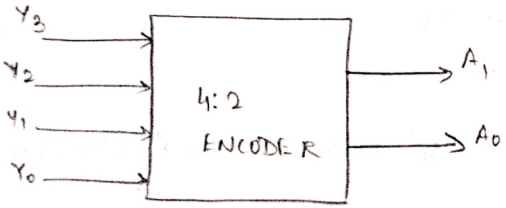
**EQUIPMENT REQUIRED:**
Bread board, connecting wires, Power supply (+5V), LED display board.
IC- 7408, 7486, 7404, 7432 etc.

**THEORY:**

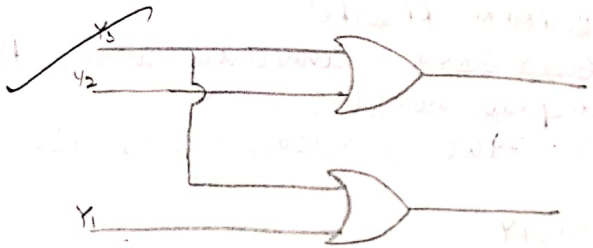
→ **ENCODER:**
Encoders are digital ICs used for encoding. By encoding, we mean generating a digital binary code for every input. An encoder IC generally consists of an enable pin which is usually set high to indicate the working. It consists of $2^n$ I/P lines and n O/P lines with each I/P line being represented by a code of zeros and ones which are reflected at the O/P line.

→ **DECODER:**
Decoders are digital ICs which are used for decoding. In other words, the decoders decrypt or obtain the actual data from the received code, i.e., convert the binary I/P at its I/P to a form, which is reflected at its O/P. It consists of n I/P lines and $2^n$ O/P lines. A decoder can be used to obtain

## 4x2 ENCODER




LOGIC SYMBOL




LOGIC CIRCUIT

$A_1 = Y_3 + Y_2$

$A_0 = Y_3 + Y_1$

Truth Table:

| INPUTS | | | | OUTPUTS | |
|---|---|---|---|---|---|
| $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ | $A_1$ | $A_0$ |
| 0 | 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 |

DATE
PAGE 16

the required data from the code or vice versa can also be used for obtaining the parallel data from the serial data received.

PROCEDURE:

1. Connections are given as per circuit diagram.
2. Logical ~~o~~ I/Ps are given as per circuit diagram.
3. Observe the O/P and verify truth table

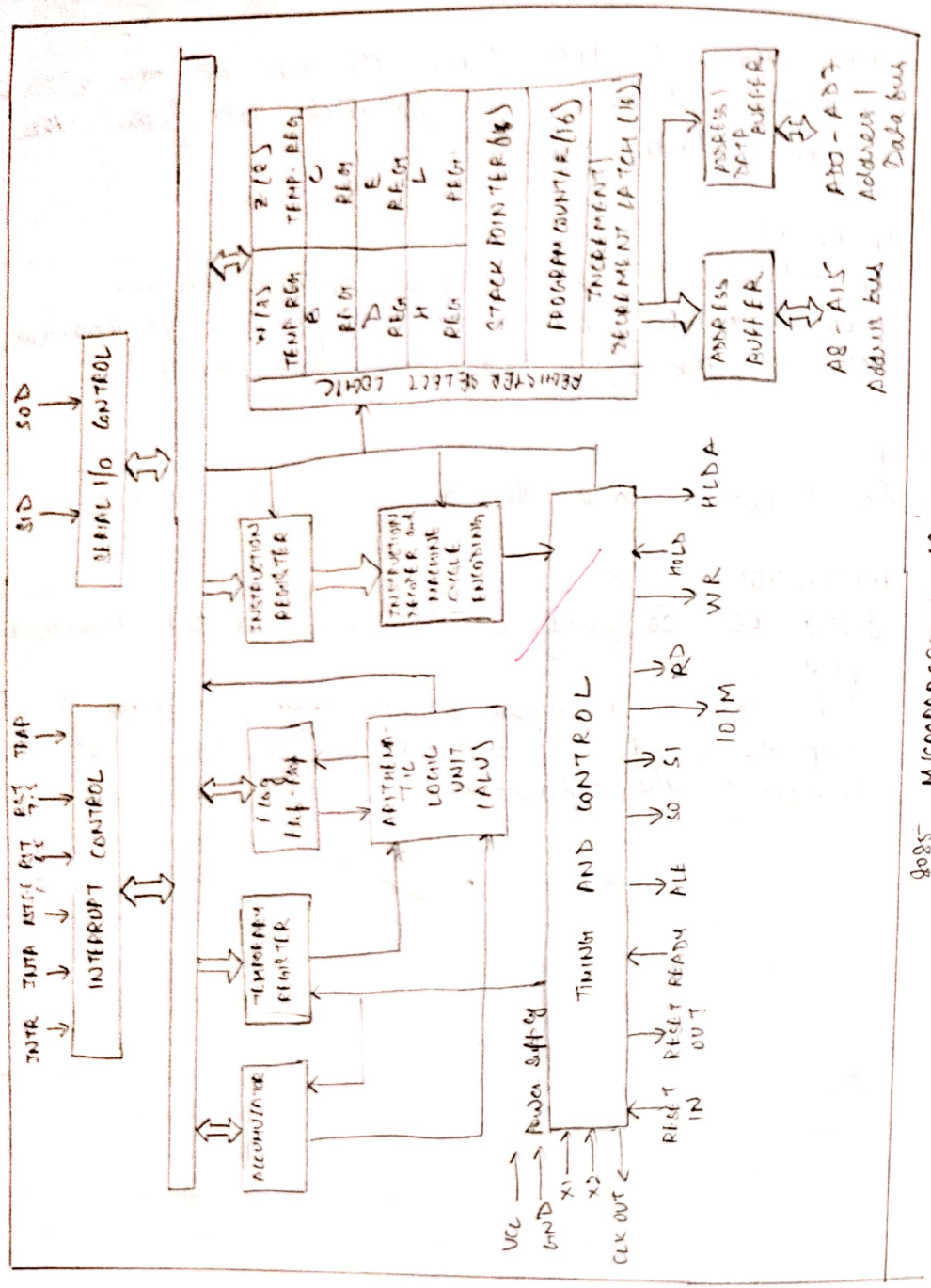
RESULT:

The Encoder and decoder designs are verified.

PRECAUTIONS:

1. Insert IC carefully in the base without damaging the pins.
2. Take care while applying the voltages to ~~the~~ the IC.
3. Keep the mind I/P and O/P pairs of the IC while performing the experiment.

Scanned with OKEN Scanner




EXP-07

DATE 14/10/25
PAGE 17

# Experiment No. 07

## OBJECTIVE:

Design a computer architecture: Design a processor with minimum number of instructions, so that it can do the basic arithmetic and logic operations.

## THEORY:

Microprocessor unit is synonymous to CPU, used in traditional computer. Microprocessor (MPU) acts as a device or a group of devices which do the following tasks.

1. Communicate with peripherals devices.
2. Provide timing signal
3. Direct of data flow
4. Perform computer tasks as specified by the ~~code~~ instruction in memory.

The 8085 microprocessor is an 8-bit general purpose microprocessor which is capable to address 64KB memory. This processor has forty pins, requires +5V single power supply and a 3-MHz single-phase clock. The related units are listed below:

## ACCUMULATOR -

In Intel 8085 microprocessor, accumulator acts as an 8-bit register to, store 8-bit data, to perform arithmatic & logical operation on them. The final result is stored in the accumulator. It is also called 'A' register

## ARITHEMATIC LOGIC UNIT (ALU)

It basically performs 8-bit of arithematic operations +, -, *, / and to perform logical operations like AND, OR, NOT, etc. ALU gets inputs from the accumulator and temporary register on which it performs such operations. After processing the necessary operations, the result stored back in the accumulator.

## TIMING AND CONTROL UNIT

This unit is responsible for generating timing signals and also it signals both inside and outside activities for Intel 8085 microprocessor. Timing and control unit is very important as it synchronises the registers and flow of data through various registers and other units. It consists of an oscillator and controller sequencer which sends control signals needed for internal and external control of data and other units. Following are the timing and control signals -

Control signals - READY, RD', WR', ALE
Status signals - S0, S1, IO/M'
DMA Signals - HOLD, HLDA
RESET signals - RESET IN, RESET OUT

## GENERAL PURPOSE REGISTER

B, C, D, E, H and L are six general purpose registers are there in 8085 microprocessor. Each register can hold 8-bit data. They can work in pairs such as BC, DE, and HL to store 16-bit data where HL pair work as a

Scanned with OKEN Scanner

memory pointer.

PROGRAM COUNTER

Program counter is a 16-bit register used to store the memory address location of the next instruction to be executed. Microprocessor increments the program whenever an instruction is being executed so that the program counter points to the memory address of the next instruction that is going to be executed.

STACK POINTER

The stack pointer is also a 16-bit register. It is used as a memory pointer. Stack pointer gets incremented when data is retrieved from the stack which is called pop operation and decremented when data is loaded into the stack that is known as push operation.

INCREMENT / DECREMENT REGISTER

The 16-bit increment and decrement registers are used to increment and decrement the PC value and SP value by 1. Increment or decrement can be performed on any register or a memory location.

ADDRESS AND ADDRESS DATA BUFFER

The content of the program counter and stack pointer are loaded into address data buffer and address buffer. These buffers are bi-directional when used

Scanned with OKEN Scanner

for data. The CPU can exchange the desired data with the memory and I/O chips as the memory and I/O chips are connected to these buses. The address data buffer is connected to both external and internal bus which consists of 8-bits

TEMPORARY REGISTER

Temporary register is an 8-bit register, which holds the temporary data of arithmetic and logical operations. The register can only be accessed by the microprocessor and it is completely inaccessible to programmer.

STATUS OR FLAG REGISTER

This register is actually a group of flip-flops to give the status of different operations result. When ALU performs an operation, the result is transferred to the internal data bus and the status of the result is stored in flag flip flops. These are set of 5 flip-flops

- Sign (S): If MSB bit = 0 then number is positive, else negative.
- Auxillary Carry (AC): If an operation performed in ALU generates the carry from lower nibble ($D_0$ to $D_3$) to upper nibble ($D_4$ to $D_7$) AC flag is set, else reset.
- Parity (P): If the result contain even number of ones, flag is set and for odd numbers of ones, flag is reset.

Scanned with OKEN Scanner

- Carry (C) : If an operation performed in ALU generates carry from D7 to next stage then CY flag is set, else reset.

INTERRUPT CONTROL

When a microprocessor is executing the main program and whenever an interrupt occurs, the microprocessor shifts the control from main program to process the incoming request. After the request is completed, the control goes back to the main program.

→ Interrupt signals present in 8085 are
INTR, RST 7.5, RST 6.5, RST 5.5, TRAP

SERIAL I/P / O/P CONTROL

It controls the serial data comm. by using these two instructions. SID (serial I/P data) and SOD (Serial O/P data)

INSTRUCTION REGISTER AND DECODER

The instruction register is also 8-bit register, the instruction register are specifically to store the instructions like adding two data's / moving a data, copying a data etc. that are fetched from memory.

RESULT :

Design of computer arithmetic operations architecture is completed.

Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| √ 0000 | | MVI A,32 | 3E | 2 | 2 | 7 |
| 0001 | | | 32 | | | |
| √ 0002 | | STA 2000 | 32 | 3 | 4 | 13 |
| 0003 | | | 00 | | | |
| 0004 | | | 20 | | | |
| √ 0005 | | HLT | 76 | 1 | 2 | 5 |

Registers | Memory | Devices

Registers :

| Register | Value | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|
| Accumulator | 32 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| Register B | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register C | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register D | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register E | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register H | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register L | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Memory(M) | 3E | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |

| Resister | Value | S | Z | * | AC | * | P | * | CY |
|---|---|---|---|---|---|---|---|---|---|
| Flag Resister | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| Type | Value |
|---|---|
| Stack Pointer(SP) | 0000 |
| Memory Pointer (HL) | 0000 |
| Program Status Word(PSW) | 3200 |
| Program Counter(PC) | 0005 |
| Clock Cycle Counter | 25 |
| Instruction Counter | 3 |

| SOD | SID | INTR | TRAP | R7.5 | R6.5 | R5.5 |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| √ 0000 | | MVI A,09 | 3E | 2 | 2 | 7 |
| 0001 | | | 09 | | | |
| √ 0002 | | MVI B,03 | 06 | 2 | 2 | 7 |
| 0003 | | | 03 | | | |
| √ 0004 | | SUB B | 90 | 1 | 1 | 4 |
| √ 0005 | | STA 3000 | 32 | 3 | 4 | 13 |
| 0006 | | | 00 | | | |
| 0007 | | | 30 | | | |
| √ 0008 | | HLT | 76 | 1 | 2 | 5 |

Registers | Memory | Devices

Registers :

| Register | Value | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|
| Accumulator | 06 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| Register B | 03 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| Register C | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register D | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register E | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register H | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register L | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Memory(M) | 3E | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |

| Resister | Value | S | Z | * | AC | * | P | * | CY |
|---|---|---|---|---|---|---|---|---|---|
| Flag Resister | 14 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |

| Type | Value |
|---|---|
| Stack Pointer(SP) | 0000 |
| Memory Pointer (HL) | 0000 |
| Program Status Word(PSW) | 0614 |
| Program Counter(PC) | 0008 |
| Clock Cycle Counter | 36 |
| Instruction Counter | 5 |

Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| √ 0000 | | MVI A,09 | 3E | 2 | 2 | 7 |
| 0001 | | | 09 | | | |
| √ 0002 | | MVI B,03 | 06 | 2 | 2 | 7 |
| 0003 | | | 03 | | | |
| √ 0004 | | SUB B | 90 | 1 | 1 | 4 |
| √ 0005 | | STA 3000 | 32 | 3 | 4 | 13 |
| 0006 | | | 00 | | | |
| 0007 | | | 30 | | | |
| √ 0008 | | HLT | 76 | 1 | 2 | 5 |

Registers | Memory | Devices

Memory Editor

Memory Range: 0000 --- FFFF

| Memory Address | Value |
|---|---|
| 0000 | 3E |
| 0001 | 09 |
| 0002 | 06 |
| 0003 | 03 |
| 0004 | 90 |
| 0005 | 32 |
| 0007 | 30 |
| 0008 | 76 |
| 3000 | 06 |

EXP-08

DATE 04/11/2025
PAGE 22

# Experiment No. 08

AIM:
Write an assembly language code in 8085 microprocessor to implement data transfer instructions.

PROGRAM:
1. MVI A, 32H : Move 32H in the accumulator.
STA 2000H : Store accumulator content at 2000H.
HLT

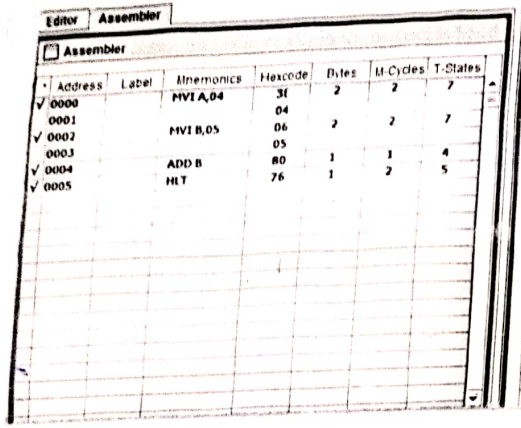
2. LXI H, 2000H : Load H-L pair by 2000H
MOV B,M : move content to register B
HLT : Terminate program execution

PROGRAM:
1. Instructions are provided as per the program.
2. Program is assembled and executed

PRECAUTIONS:
1. All instructions should be according to program.
2. Instructions operations should be pre-learned.

RESULT:
Operation of data transfer instructions is verified

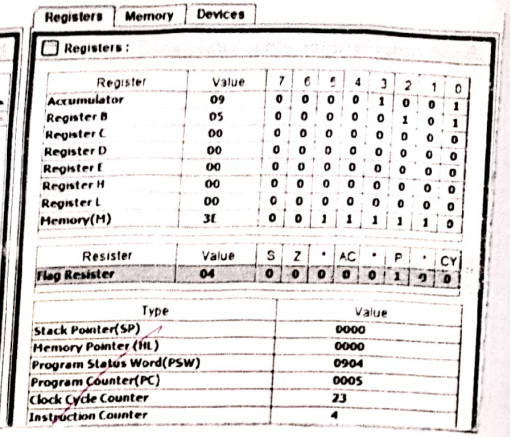
Scanned with OKEN Scanner



Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| √ 0000 | | MVI A,04 | 3E | 2 | 2 | 7 |
| 0001 | | | 04 | | | |
| √ 0002 | | MVI B,05 | 06 | 2 | 2 | 7 |
| 0003 | | | 05 | | | |
| √ 0004 | | ADD B | 80 | 1 | 1 | 4 |
| √ 0005 | | HLT | 76 | 1 | 2 | 5 |



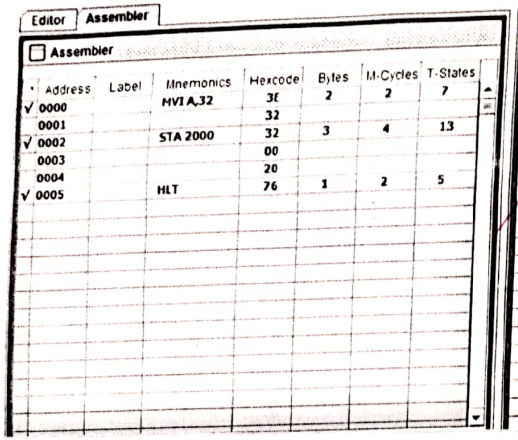
Registers | Memory | Devices

Registers :

| Register | Value | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|
| Accumulator | 09 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| Register B | 05 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| Register C | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register D | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register E | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register H | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register L | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Memory(M) | 3E | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 |

| Resister | Value | S | Z | · | AC | · | P | · | CY |
|---|---|---|---|---|---|---|---|---|---|
| Flag Resister | 04 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

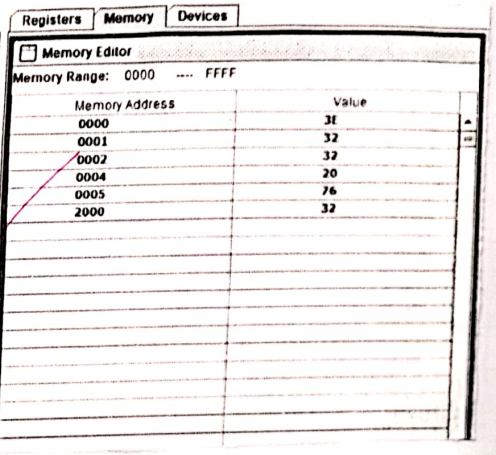
| Type | Value |
|---|---|
| Stack Pointer(SP) | 0000 |
| Memory Pointer (HL) | 0000 |
| Program Status Word(PSW) | 0904 |
| Program Counter(PC) | 0005 |
| Clock Cycle Counter | 23 |
| Instruction Counter | 4 |



Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| √ 0000 | | MVI A,32 | 3E | 2 | 2 | 7 |
| 0001 | | | 32 | | | |
| √ 0002 | | STA 2000 | 32 | 3 | 4 | 13 |
| 0003 | | | 00 | | | |
| 0004 | | | 20 | | | |
| √ 0005 | | HLT | 76 | 1 | 2 | 5 |

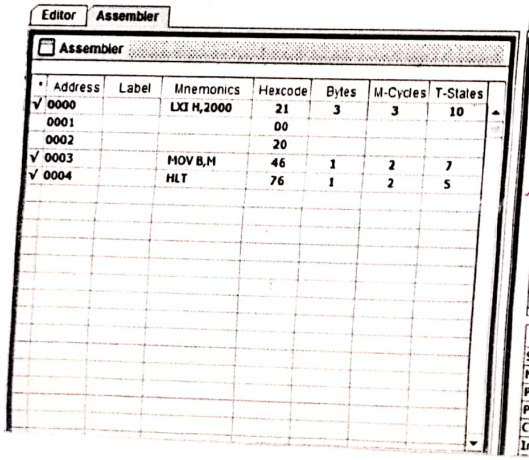


Registers | Memory | Devices

Memory Editor

Memory Range: 0000 ---- FFFF

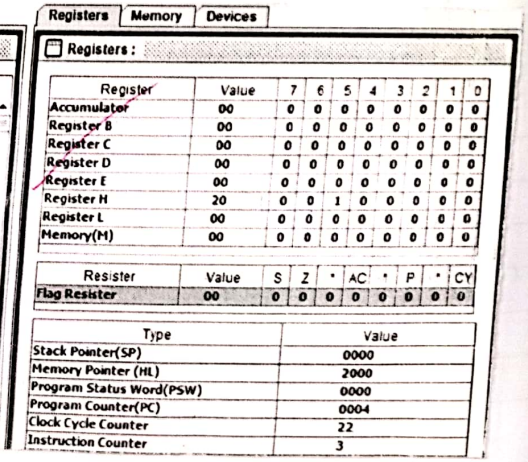
| Memory Address | Value |
|---|---|
| 0000 | 3E |
| 0001 | 32 |
| 0002 | 32 |
| 0004 | 20 |
| 0005 | 76 |
| 2000 | 32 |



Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| √ 0000 | | LXI H,2000 | 21 | 3 | 3 | 10 |
| 0001 | | | 00 | | | |
| 0002 | | | 20 | | | |
| √ 0003 | | MOV B,M | 46 | 1 | 2 | 7 |
| √ 0004 | | HLT | 76 | 1 | 2 | 5 |



Registers | Memory | Devices

Registers :

| Register | Value | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|
| Accumulator | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register B | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register C | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register D | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register E | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register H | 20 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| Register L | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Memory(M) | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| Resister | Value | S | Z | · | AC | · | P | · | CY |
|---|---|---|---|---|---|---|---|---|---|
| Flag Resister | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

| Type | Value |
|---|---|
| Stack Pointer(SP) | 0000 |
| Memory Pointer (HL) | 2000 |
| Program Status Word(PSW) | 0000 |
| Program Counter(PC) | 0004 |
| Clock Cycle Counter | 22 |
| Instruction Counter | 3 |

EXP-09

DATE 04/11/2025
PAGE 23

## Experiment No. 09.

AIM:

Write an assembly language code in GNU simulator to implement arithmatic instruction

PROGRAM:

1. MVI A, 04H : Move 04H in accumulator
   MVI B, 05H : Move 05H in register B
   ADD B : Add accumulator and B content, store in A
   HLT : Terminate program execution.

2. MVI A, 09H : Move 09H in accumulator
   MVI B, 03H : Move 03H in register B
   SUB B : Subtract A and B content and store in A.
   STA 3000H : Store accumulator content at 3000H
   HLT : Terminate program execution

PROCEDURE :

1. Instructions are provided as per program.
2. Instruction should be pre learned.

RESULT :

Operation of arithmatic instructions is verified.

Scanned with OKEN Scanner

Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| 0000 | | LXI H,6000 | 21 | 3 | 3 | 10 |
| 0001 | | | 00 | | | |
| 0002 | | | 60 | | | |
| 0003 | | MOV A,M | 7E | 1 | 2 | 7 |
| 0004 | | INX H | 23 | 1 | 1 | 6 |
| 0005 | | ADD M | 86 | 1 | 2 | 7 |
| 0006 | | STA 6002 | 32 | 3 | 4 | 13 |
| 0007 | | | 02 | | | |
| 0008 | | | 60 | | | |
| 0009 | | HLT | 76 | 1 | 2 | 5 |

Registers | Memory | Devices

Registers :

| Register | Value | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|
| Accumulator | 03 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| Register B | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register C | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register D | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register E | 00 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| Register H | 60 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| Register L | 01 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| Memory(M) | 02 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |

| Register | Value | S | Z | * | AC | * | P | * | CY |
|---|---|---|---|---|---|---|---|---|---|
| Flag Register | 04 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

| Type | Value |
|---|---|
| Stack Pointer(SP) | 0000 |
| Memory Pointer (HL) | 6001 |
| Program Status Word(PSW) | 0304 |
| Program Counter(PC) | 0009 |
| Clock Cycle Counter | 101 |
| Instruction Counter | 13 |

Editor | Assembler

Assembler

| Address | Label | Mnemonics | Hexcode | Bytes | M-Cycles | T-States |
|---|---|---|---|---|---|---|
| 0000 | | LXI H,6000 | 21 | 3 | 3 | 10 |
| 0001 | | | 00 | | | |
| 0002 | | | 60 | | | |
| 0003 | | MOV A,M | 7E | 1 | 2 | 7 |
| 0004 | | INX H | 23 | 1 | 1 | 6 |
| 0005 | | ADD M | 86 | 1 | 2 | 7 |
| 0006 | | STA 6002 | 32 | 3 | 4 | 13 |
| 0007 | | | 02 | | | |
| 0008 | | | 60 | | | |
| 0009 | | HLT | 76 | 1 | 2 | 5 |

Registers | Memory | Devices

Memory Editor

Memory Range: 0000 --- FFFF

| Memory Address | Value |
|---|---|
| 0000 | 21 |
| 0002 | 60 |
| 0003 | 7E |
| 0004 | 23 |
| 0005 | 86 |
| 0006 | 32 |
| 0007 | 02 |
| 0008 | 60 |
| 0009 | 76 |
| 6000 | 01 |
| 6001 | 02 |
| 6002 | 03 |

EXP-10

DATE 04/11/2025
PAGE 24

## Experiment No. 10

AIM:
Write an assembly language code in GNU sim 8085 to add two 8-bit number using memory.

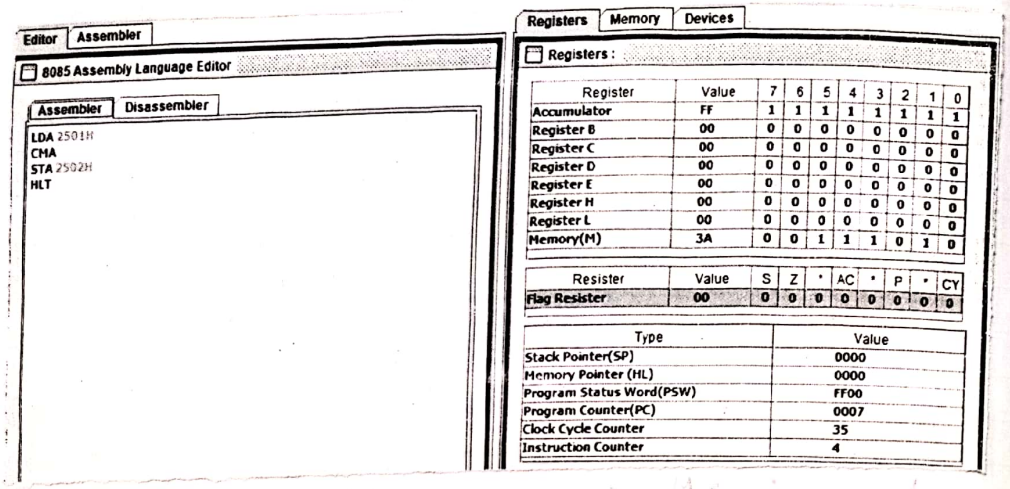
PROGRAM:
LXI H, 6000 H : Get address of first number in H-L pair
MOV A, M : Get number in accumulator
INX H : Increment content of H-L pair
ADD M : Add both numbers.
STA 6002 H : Store sum in 6002 H
HLT : Terminate program execution

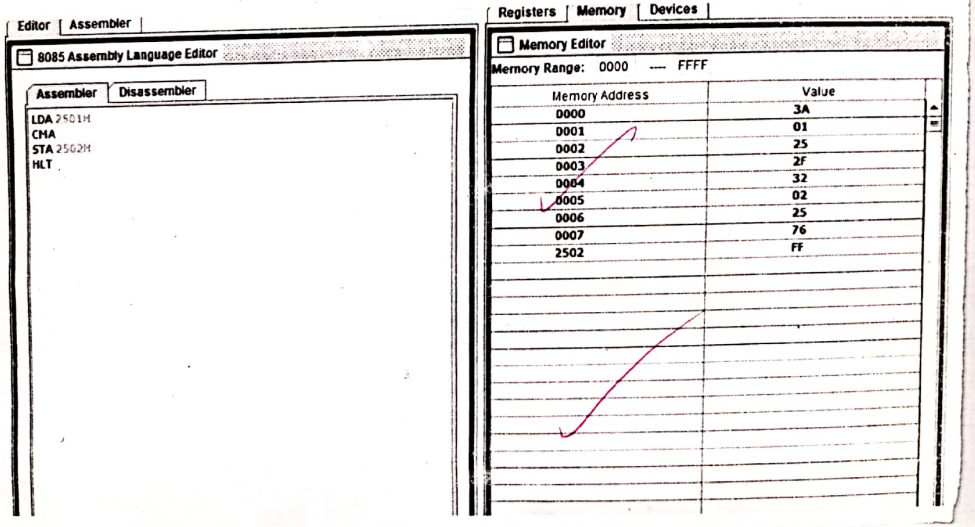
PROCEDURE:
1. Instructions are provided as per program
2. Program is assembled and executed.

PRECAUTIONS:
1. The instruction should be according to program.
2. Instruction operation should be pre-learned.

RESULT:
The operation of two 8-bit numbers addition is verified.

Scanned with OKEN Scanner




EXP-11

DATE 04/11/2018
PAGE 25

## Experiment No. 11

AIM:

Write an assembly language in GNU Sim 8085 to implement logical instruction.

PROGRAM:

LDA 2501H: Get data into accumulator
CMA: Take its complement
STA 2502H: Store result in 2502H
HLT: Terminate program execution

PROCEDURE:

1. Instructions are provided as per program.
2. Program is assembled and executed.

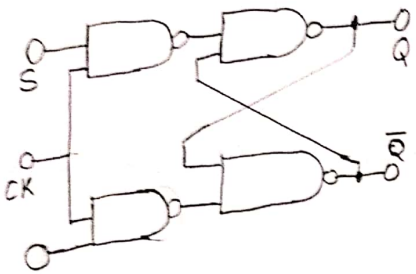
PRECAUTIONS:

1. All instructions should be acc. to program.
2. Instruction operation should be pre-learned.

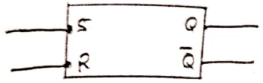
RESULT:

Operation of logical instruction is verified.

Scanned with OKEN Scanner




Logic Circuit




Logic Symbol

Truth Table

| INPUTS | | | OUTPUTS | REMARKS |
|---|---|---|---|---|
| S | R | Qn (Present State) | Qn+1 (Next State) | States and condition |
| 0 | 0 | X | Qn | Hold State condition S=R=0 |
| 0 | 1 | X | 0 | Reset State condition S=0 R=1 |
| 1 | 0 | X | 1 | Set State condition S=1 R=0 |
| 1 | 1 | X | Indeterminate | Indeterminate State condition S=R=1 |

EXP - 12.

DATE 11/11/2015
PAGE 26

## Experiment No: 12

AIM:

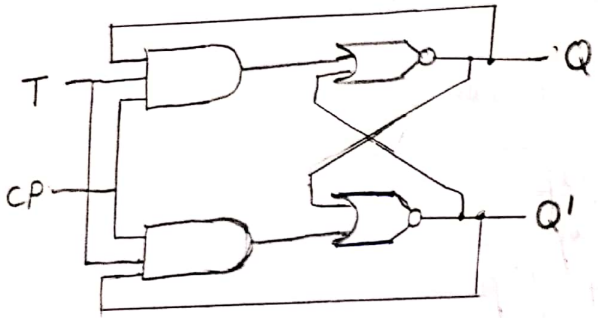
Truth Table verification of
(i) SR flip flop
(ii) D type flip flop
(iii) T type flip flop
(iv) JK type flip flop

EQUIPMENT REQUIRED:

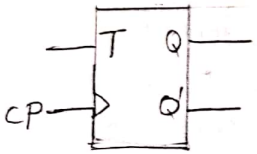
| S.NO. | NAME OF COMPONENT | SPECIFICATIONS | QUANTITY |
|---|---|---|---|
| 1. | Digital Trainer kit | DL-AD TK | 1 |
| 2. | IC 7400, 7402 | NAND, NOR | 4 |
| 3. | IC 7474, IC 7476 | D FF, JK FF | 1 each |
| 4. | IC 7404, 7408 | NOT, AND | 4 |
| 5 | IC 7432, 7486 | OR, EX-OR | 4 |
| 6. | Patch cords/Connecting wires | – | As per req'd. |

THEORY:

Logic circuits that incorporate memory cells are called sequential logic circuits; their O/P depends not only upon the present value of the I/P but also upon the previous values. Sequential logic circuits often require a timing generator (or clock) for their operation. The latch (flip flop) is a basic bi-stable memory element widely used in sequential logic circuits. Usually, there are 2 O/Ps




@ Logic diagram




b. Graphical Symbol

| Q | T | Q(t+1) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Transition table

Clocked T flip-flop

DATE
PAGE

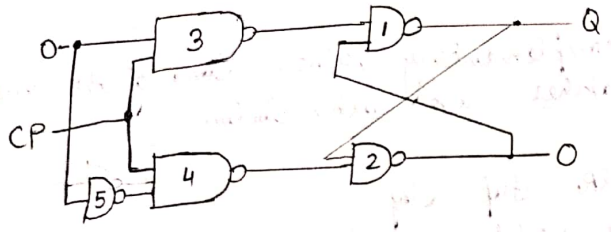
S and its complementary value. Some of the most widely used latches are listed below:

(i) The clocked SR Flip-Flop

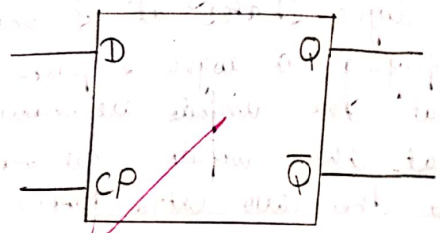
Fig 1, shows a useful variation on the basic SR flip flop the clocked SR flip-flop. By adding two extra NAND gates, the timing of the O/P changeover after a change of logic states at S and R can be controlled by applying a logic 1 pulse to the clock (CK) input. Note that the inputs are now labelled S and R indicating that the inputs are now 'high activated'. This is because the two extra NAND gates are disabled while the CK input is low, therefore O/P are completely isolated from the inputs and to retain any previous logic state, but when the CK input is high (during a clk pulse) the I/P NAND gates act as inverters. Thus for example, a logic 1 applied to the S I/P of the active low SR flip-flop second stage clks.

(ii) T flip flop

This is much simpler version of the JK flip flop. Both the J and K I/Ps are connected together, and thus are also called single input J-K flip flop. When clk pulse is given to the flip-flop an the O/P begins to toggle. Here also the restriction on the pulse width can be eliminated with a master-slave or edge-triggering construction. Take a look at the circuit and truth table below




Logic Circuit




Logic Symbol

Truth Table

| S | D | QN+1 |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

DATE
PAGE

(iii) D flip flop

The flip flop is a basic building block of sequential logic ckts. It is a circuit that has two stable states and can store one bit of state info. The O/P changes state by signals applied to one or more control I/Ps. The basic D flip flop has a D (data) input and a clock input and O/Ps Q and $\bar{Q}$ (the inverse of Q). Optionally it may also include the PR (Preset) and CLR (Clear) control I/Ps. The truth table and diagram.

(iv) JK flip flop

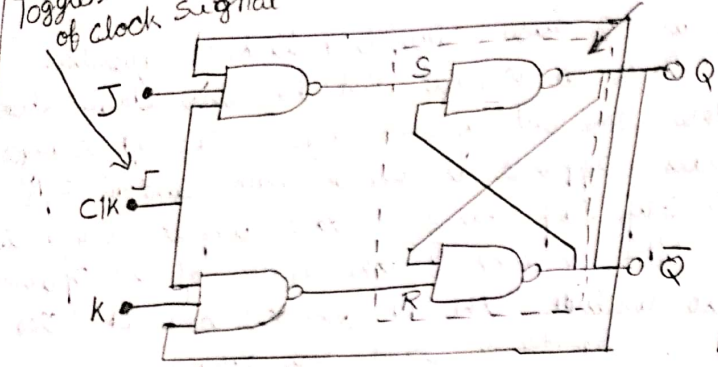
Both the S and the R I/Ps of the previous SR latch have now been replaced by two I/Ps called the J and K I/Ps, respectively after its inventor Jack Kilby. Then this equates to: J=S and K=R. The two 2-input AND gates of the gated SR latch have now been replaced by two 3-I/P NAND gates with the third I/P of each gate connected to the O/Ps at Q and $\bar{Q}$. This cross coupling of the SR flip-flop allows the previously obtained condition of S = "1" and R = "1" state to be used to produce a "toggle action" as the two I/Ps are now interlocked.

If the ckt is now "SET" the J input is inhibited by the "0" status of $\bar{Q}$ through the upper NAND gate. As Q and $\bar{Q}$ are always opposite, we can use them to control the I/P.

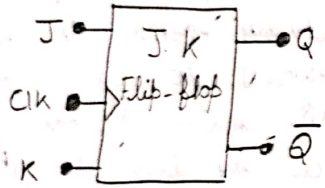
Scanned with OKEN Scanner

Toggles on leading edge of clock signal

SR flip-flop




Circuit




Symbol

DATE
PAGE

PROCEDURE:

1. Check all the comps. for their working
2. Insert the appropriate IC into the IC base.
3. Make connections as shown in the ckt. diagram.
4. Verify the Truth table and observe the o/ps.

PRECAUTIONS:

1. The connections should be acc. to ckt. diagram
2. All connections should be right and tight
3. Power supply switched OFF after completing the experiment.

RESULT:

All the flip flops have been implemented & the truth table is verified.

Scanned with OKEN Scanner